Конспект: модуль SystemTokenAuth — детальный разбор

Модуль реализует **привязку авторизации к конкретному компьютеру**: пользователь может «запомнить этот ПК», и при следующем запуске приложение автоматически считает его авторизованным — но только на этом же ПК и под этой же учётной записью Windows.

---

1. Назначение и контекст

Проблема/Контекст

В приложении ShifRator3 нужна возможность «Войти на этом компьютере» (аналог «Запомнить меня» в веб-формах). При этом:

- Токен не должен действовать на другом ПК — иначе копирование одного файла дало бы доступ с любого компьютера.
- Токен не должен действовать под другой учётной записью Windows на том же ПК — иначе один файл открывал бы доступ всем пользователям машины.

Решение

В файл сохраняются два значения: **отпечаток текущей системы** (fingerprint) и **имя пользователя приложения**. Отпечаток вычисляется из стабильных данных Windows (серийный номер тома C:\, имя компьютера, имя пользователя ОС). При следующем запуске отпечаток вычисляется снова и сравнивается с сохранённым. Совпадение означает «это тот же ПК и тот же пользователь ОС» — тогда считаем пользователя приложения авторизованным под сохранённым именем.

> **Ключевая идея:** отпечаток — это хеш от строки вида `volSerial|compName|userName`. На другом ПК или под другим пользователем Windows эта строка будет другой, хеш изменится, и сохранённый токен не пройдёт проверку.

---

2. Подключаемые заголовки

Зачем каждый include?

Файл	Назначение
`SystemTokenAuth.h`	Объявления функций модуля и тип возврата `std::pair<bool, std::string>`.
`windows.h`	GetVolumeInformationW, GetComputerNameW, GetUserNameW, GetTempPathA, CreateDirectoryW, DeleteFileA, DWORD, MAX_PATH и т.д.
`shlobj.h`	SHGetFolderPathW и константа CSIDL_LOCAL_APPDATA.
`fstream`	std::ifstream, std::ofstream для чтения/записи файла токена.
`sstream`	std::ostringstream для сборки строк (например, число в строку, hex).
`iomanip`	std::setfill, std::setw для форматирования hex-строки с ведущими нулями.
`vector`	Не используется напрямую в показанном коде; может использоваться в алгоритмах или при доработках.
`algorithm`	std::remove_if при очистке строк от символов `\r` и `\n`.

---

3. Анонимный namespace и константы

Синтаксис `namespace { ... }`

Всё, что объявлено внутри `namespace { ... }`, имеет **внутреннюю линковку**: эти имена видны только в данном .cpp файле. С другими единицами трансляции они не конфликтуют. Функции `WideToNarrow`, `Fnv1aHash`, `ToHex` и константы `TOKEN_FILENAME`, `TOKEN_SEP` — детали реализации, в заголовке они не объявляются.

Константы

- `TOKEN_FILENAME = L"ShifRator3_system_token.dat"` — имя файла в широких символах (L""), т.к. путь потом собирается из широких строк Windows API.
- `TOKEN_SEP = '\n'` — разделитель первой и второй строки в файле. Первая строка — отпечаток, вторая — имя пользователя. `std::getline(f, s, '\n')` читает до

первого `\n`.

---

4. WideToNarrow — преобразование wstring → string (UTF-8)

Проблема/Контекст

Функции Windows API (GetComputerNameW, GetUserNameW и т.д.) возвращают строки в виде `wchar_t*` (UTF-16 под Windows). Для записи в текстовый файл и для передачи в Fnv1aHash нужна байтовая строка. В коде используется UTF-8: одна и та же строка даёт одни и те же байты независимо от локали, и при необходимости файл можно просмотреть в любом редакторе.

Сигнатура и пошаговый разбор

```
std::string WideToNarrow(const std::wstring& ws)
{
        if (ws.empty()) return {};
        int len = WideCharToMultiByte(CP_UTF8, 0, ws.c_str(), (int)ws.size(), nullpt
        if (len <= 0) return {};
        std::string s(static_cast(len), '\0');
        WideCharToMultiByte(CP_UTF8, 0, ws.c_str(), (int)ws.size(), &s[0], len, nul
        return s;
}
```

Параметры WideCharToMultiByte (первый вызов)

Объявление (упрощённо):

```
int WideCharToMultiByte(
    UINT   CodePage,     // CP_UTF8 = 65001
    DWORD  dwFlags,      // 0 — обычное преобразование
    LPCWCH lpWideCharStr,
    int    cchWideChar,
    LPSTR  lpMultiByteStr,  // nullptr — только посчитать длину
    int    cbMultiByte,     // 0
    LPCCH  lpDefaultChar,   // nullptr
    LPBOOL lpUsedDefaultChar // nullptr
);
```

- `CP_UTF8` (65001) — целевая кодовая страница: на выходе получаем UTF-8.

- `ws.c_str()` — указатель на широкие символы; `(int)ws.size()` — количество широких символов (не байт). Приведение нужно, т.к. API принимает `int`.
- `lpMultiByteStr = nullptr` и `cbMultiByte = 0` — буфер не передаём. Функция возвращает **нужное количество байт** для результата (включая завершающий нуль, если он запрашивается; при нулевом буфере возвращаемое значение — размер без нуля или с ним в зависимости от версии — в данном использовании это «достаточный размер»).

В коде возвращаемое значение трактуется как «сколько байт выделить». Если `len <= 0`, преобразование не удалось (например, неверные данные) — возвращаем пустую строку.

Второй вызов WideCharToMultiByte

Создаётся строка `s` из `len` символов `'\0'`. Затем тот же вызов, но с буфером `&s[0]` и размером `len`: функция заполняет буфер байтами UTF-8. Результат не проверяется повторно: при успешном первом вызове второй обычно тоже успешен. Строка `s` содержит ровно `len` байт (без завершающего нуля в размере; для `std::string` это нормально).

> Важно: в C++11 и новее запись в `&s[0]` для непустой `std::string` допустима и не меняет размер строки. Размер остаётся `len`; фактическое содержимое — то, что записал WideCharToMultiByte. Если бы API дописывал нуль-терминатор, он мог бы оказаться внутри строки; в данном использовании длина подобрана так, что записывается ровно нужное количество байт.

---

5. Fnv1aHash — детерминированный хеш (FNV-1a)

Зачем хеш?

Отпечаток системы должен быть коротким и фиксированной длины, но однозначно зависеть от строки `volSerial|compName|userName`. Хеш-функция превращает строку произвольной длины в 64 бита. FNV-1a — быстрая, простая, детерминированная; криптостойкость не требуется, нужна лишь устойчивость к случайным изменениям данных.

Код с пояснениями по строкам

```
uint64_t Fnv1aHash(const std::string& s)
{
        uint64_t h = 14695981039346656037ULL; // FNV offset basis для 64-bit
        for (unsigned char c : s)
                h = (h ^ c) * 1099511628211ULL;   // FNV prime для 64-bit
        return h;
}
```

- **14695981039346656037ULL** — стандартная константа FNV offset basis для 64-битного варианта. Начальное значение «нейтральное» для алгоритма.
- **unsigned char c** — обходим строку по байтам. Приведение к `unsigned char` гарантирует неотрицательное значение 0..255 и одинаковое поведение на платформах с signed char.
- **h = (h ^ c) * 1099511628211ULL** — один шаг FNV-1a: сначала XOR текущего хеша с байтом, затем умножение на FNV prime (1099511628211). Арифметика беззнаковая, переполнение идёт по модулю 2^{64} — это предусмотрено стандартом.

Пример трассировки (упрощённо)

Пусть `s = "A"` (один байт 0x41). Тогда:

- `h0 = 14695981039346656037` ;
- `h1 = (h0 ^ 0x41) * 1099511628211` — одно число, всегда одно и то же для `"A"` .

Для любой другой строки (например, `"123|DESKTOP-ABC|Ivan"`) результат будет другой, но для той же строки — всегда одинаковый.

> Итог: одна и та же строка `raw` всегда даёт один и тот же `uint64_t` , что и нужно для сравнения отпечатков.

---

6. ToHex — число в 16-ричную строку фиксированной длины

Назначение

Хеш — значение типа `uint64_t` . Его нужно сохранять в файл и сравнивать как строку. Перевод в hex даёт ровно 16 символов (64 бита / 4 = 16 hex-цифр). Фиксированная длина удобна при чтении и разборе файла.

```
std::string ToHex(uint64_t v)
{
        std::ostringstream os;
        os << std::hex << std::setfill('0') << std::setw(16) << v;
        return os.str();
}
```

Разбор манипуляторов

- `std::hex` — вывод в шестнадцатеричном формате (цифры 0–9, a–f).
- `std::setfill('0')` — символ заполнения при недостаточной ширине — ноль.
- `std::setw(16)` — минимальная ширина поля 16 символов. Если число даёт меньше 16 hex-цифр (например, 0 → `"0"`), слева дополняется нулями: `0000000000000000`.

Пример: `v = 0x1234` → строка `"0000000000001234"`. Так отпечаток в файле всегда имеет длину 16 символов.

---

7. GetSystemFingerprint() — построчное описание

Идея

Собираем строку `raw = volSerial + "|" + compName + "|" + userName`, затем `return ToHex(Fnv1aHash(raw))`. Любой из трёх компонентов может быть пустым при ошибке API (тогда в `raw` останутся только разделители или часть данных).

Блок 1: серийный номер тома C:\

```
DWORD volSerial = 0, maxLen = 0;
wchar_t volName[MAX_PATH + 1] = {};
if (GetVolumeInformationW(L"C:\\", volName, MAX_PATH, &volSerial, &maxLen, nullptr,
{
        std::ostringstream os;
        os << volSerial;
        raw += os.str();
}
raw += "|";
```

GetVolumeInformationW — объявление (упрощённо):

```
BOOL GetVolumeInformationW(
    LPCWSTR lpRootPathName,   // L"C:\\" — корень тома
    LPWSTR  lpVolumeNameBuffer,
    DWORD   nVolumeNameSize,  // MAX_PATH
    LPDWORD lpVolumeSerialNumber,  // сюда пишется серийный номер
    LPDWORD lpMaximumComponentLength,
    LPDWORD lpFileSystemFlags,
    LPWSTR  lpFileSystemNameBuffer,
    DWORD   nFileSystemNameSize
);
```

- `lpVolumeSerialNumber` — указатель на `volSerial`; после успешного вызова в нём серийный номер тома (например, 0xA1B2C3D4). Передаём `nullptr` для параметров, которые не нужны (`lpFileSystemFlags`, имена ФС и т.д.).
- Если вызов вернул TRUE, `volSerial` преобразуется в десятичную строку через `ostringstream` и добавляется к `raw`. Если вызов не удался (например, нет доступа к C:\), к `raw` добавляется только `"|"`.

Блок 2: имя компьютера

```
wchar_t compName[MAX_COMPUTERNAME_LENGTH + 1] = {};
DWORD size = MAX_COMPUTERNAME_LENGTH + 1;
if (GetComputerNameW(compName, &size))
        raw += WideToNarrow(compName);
raw += "|";
```

GetComputerNameW(compName, &size): `size` при вызове — размер буфера в символах (включая нуль); после вызова в `size` может быть записано реальное количество записанных символов. Результат в UTF-16 — переводим в UTF-8 через `WideToNarrow` и добавляем к `raw`. Разделитель `"|"` добавляется в любом случае.

Блок 3: имя пользователя Windows

```
wchar_t userBuf[256] = {};
DWORD userLen = 256;
if (GetUserNameW(userBuf, &userLen))
        raw += WideToNarrow(userBuf);

uint64_t h = Fnv1aHash(raw);
return ToHex(h);
```

GetUserNameW(userBuf, &userLen): аналогично — буфер и размер; после вызова в буфере имя текущего пользователя. Добавляем к `raw` в UTF-8 (или не добавляем при ошибке). Затем хешируем `raw` и возвращаем его hex-представление.

Пример значения raw и результата

Пусть `volSerial = 0x1A2B3C4D`, имя компьютера `DESKTOP-XYZ`, пользователь `Ivan`. Тогда:

`raw = "439041101|DESKTOP-XYZ|Ivan"` (серийный номер в десятичном виде).

После `Fnv1aHash(raw)` и `ToHex(h)` получится строка из 16 hex-символов, например `a1b2c3d4e5f67890`. На другом ПК или под другим пользователем `raw` будет другой → хеш другой → отпечаток не совпадёт.

> Если пользователь переименует компьютер, сменит том или войдёт под другой учётной записью Windows, отпечаток изменится и сохранённый токен перестанет подходить. Это задуманное поведение «привязки к ПК и пользователю ОС».

---

8. GetTokenFilePath() — путь к файлу токена

Цель

Файл должен лежать в каталоге приложения внутри профиля пользователя, чтобы разные пользователи Windows имели разные токены. Используется **Local AppData**: `%LOCALAPPDATA%\ShifRator3\ShifRator3_system_token.dat`.

```
wchar_t buf[MAX_PATH];
if (!SUCCEEDED(SHGetFolderPathW(nullptr, CSIDL_LOCAL_APPDATA, nullptr, 0, buf)))
{
        char temp[MAX_PATH];
        if (GetTempPathA(MAX_PATH, temp))
                return std::string(temp) + "ShifRator3\\";
        return "";
}
std::wstring folder = std::wstring(buf) + L"\\ShifRator3\\";
CreateDirectoryW(folder.c_str(), nullptr);
return WideToNarrow(folder) + WideToNarrow(TOKEN_FILENAME);
```

SHGetFolderPathW

- `hwnd = nullptr` — окно не используется.
- `CSIDL_LOCAL_APPDATA` — идентификатор папки «Local AppData» (например, `C:\Users\Ivan\AppData\Local`).
- `buf` — буфер, куда записывается путь в широких символах.

`SUCCEEDED(hr)` проверяет код возврата (HRESULT). Если папку получить не удалось, переходим к запасному варианту.

Запасной путь (GetTempPathA)

Если Local AppData недоступен, используется временная папка системы (`GetTempPathA` возвращает путь с завершающим `\`). Код возвращает `temp + "ShifRator3\\"` — это путь только к каталогу, без имени файла. В основной ветке ниже возвращается полный путь к файлу: `folder + TOKEN_FILENAME`. В fallback имени файла нет, поэтому при открытии `std::ifstream f(path)` в `TryAuthorizeBySystemToken` фактически передаётся путь к папке — файл открыться не сможет. Для полной корректности в fallback стоит возвращать `std::string(temp) + "ShifRator3\\" + "ShifRator3_system_token.dat"` (или использовать аналог `WideToNarrow(TOKEN_FILENAME)`).

Создание каталога и формирование пути к файлу

`folder = buf + L"\\ShifRator3\\"` — полный путь к подкаталогу ShifRator3. `CreateDirectoryW(folder.c_str(), nullptr)` создаёт каталог, если его нет; если уже есть, GetLastError() вернёт ERROR_ALREADY_EXISTS, но это не проверяется — для логики достаточно того, что каталог есть. Возвращаем `WideToNarrow(folder) + WideToNarrow(TOKEN_FILENAME)` — полный путь к файлу в виде узкой строки (UTF-8), например `C:\Users\Ivan\AppData\Local\ShifRator3\ShifRator3_system_token.dat` (в UTF-8 представлении).

> Возвращается именно путь к **файлу**, а не только к каталогу — его можно сразу передать в `std::ifstream / std::ofstream` и в `DeleteFileA`.

---

9. TryAuthorizeBySystemToken() — детальный разбор

Назначение

Вызывается при старте приложения. Читает файл токена, извлекает сохранённый отпечаток и имя пользователя, сравнивает отпечаток с текущим. Возвращает `std::pair<bool, std::string>` : успех/неуспех и имя пользователя (или пустая строка).

Шаг 1: путь и открытие файла

```
std::string path = GetTokenFilePath();
if (path.empty()) return {false, ""};

std::ifstream f(path, std::ios::in);
if (!f.is_open()) return {false, ""};
```

Если путь пустой (не удалось получить ни AppData, ни temp) или файл не открылся (нет файла, нет прав) — сразу возврат `{false, ""}` .

Шаг 2: чтение двух строк

```
std::string storedFingerprint, username;
if (!std::getline(f, storedFingerprint, TOKEN_SEP)) return {false, ""};
if (!std::getline(f, username)) { /* username может быть пустым */ }
f.close();
```

- `std::getline(f, storedFingerprint, TOKEN_SEP)` — читает из потока `f` в строку `storedFingerprint` до разделителя `TOKEN_SEP` (`'\n'`). Разделитель из потока извлекается, в строку не попадает. Если прочитать не удалось (файл пустой или только перевод строки), getline вернёт false — выходим с `{false, ""}` .
- `std::getline(f, username)` — читает вторую строку до следующего `\n` (по умолчанию разделитель — `'\n'`). Если второй строки нет (файл содержал только одну строку), `username` останется пустым, getline может вернуть false — это допускается комментарием: «username может быть пустым».

Шаг 3: очистка от \r и \n

```
storedFingerprint.erase(std::remove_if(storedFingerprint.begin(), storedFingerprint
        [](char c) { return c == '\r' || c == '\n'; }), storedFingerprint.end());
username.erase(std::remove_if(username.begin(), username.end(),
        [](char c) { return c == '\r' || c == '\n'; }), username.end());
```

std::remove_if(beg, end, pred) сдвигает «нужные» элементы в начало диапазона и возвращает итератор на новый логический конец. Сами данные после этого «конца» не стираются. Вызов `erase(newEnd, end)` удаляет хвост контейнера. В результате из строки удаляются все символы `\r` и `\n` (на случай, если файл был записан с \r\n или прочитан на другой ОС). Отпечаток должен быть ровно 16 hex-символов без переводов строк.

Шаг 4: сравнение отпечатков

```
std::string current = GetSystemFingerprint();
if (storedFingerprint != current) return {false, ""};
return {true, username};
```

Вычисляем текущий отпечаток системы. Если он не совпадает с сохранённым — токен не для этой системы или пользователя ОС → `{false, ""}`. Иначе — авторизация по токену успешна, возвращаем `{true, username}`.

> Итог: успех только при наличии файла, двух строках в нём, совпадении отпечатка и (опционально) непустом имени пользователя. Пустое имя после проверки отпечатка допустимо — приложение может трактовать его по своему усмотрению.

---

10. SaveSystemToken(username)

Когда вызывать

После успешной авторизации по логину/паролю, если пользователь включил опцию «Запомнить на этом компьютере».

```
std::string path = GetTokenFilePath();
if (path.empty()) return false;

std::ofstream f(path, std::ios::out | std::ios::trunc);
if (!f.is_open()) return false;

f << GetSystemFingerprint() << TOKEN_SEP << username << "\n";
f.close();
return true;
```

Режимы открытия

- `std::ios::out` — вывод (запись).
- `std::ios::trunc` — при открытии обрезать файл до нуля байт. Если файл существовал, старое содержимое теряется — записывается только новый отпечаток и новый username.

Порядок записи: сначала отпечаток, затем `TOKEN_SEP` (`'\n'`), затем `username`, затем ещё один `"\n"`. Так формат совпадает с тем, что читает `TryAuthorizeBySystemToken`: первая строка — отпечаток, вторая — username.

---

11. ClearSystemToken()

```
void ClearSystemToken()
{
        std::string path = GetTokenFilePath();
        if (path.empty()) return;
        DeleteFileA(path.c_str());
}
```

DeleteFileA (WinAPI) удаляет файл по пути. Возвращаемое значение не проверяется: если файла нет или нет прав — токена всё равно нет, повторный вызов TryAuthorizeBySystemToken просто не найдёт файл. Логика «удалить токен» от этого не страдает.

---

12. Сводка по использованию в приложении

Типичный сценарий

1. **Старт приложения:** вызвать `TryAuthorizeBySystemToken()`. Если вернулось `(true, username)` — считать пользователя авторизованным под `username`; иначе показать форму входа.
2. **Успешный вход по паролю + галочка «Запомнить»:** вызвать `SaveSystemToken(username)`.
3. **Выход / снять «Запомнить»:** вызвать `ClearSystemToken()`.

> **Итог:** модуль SystemTokenAuth реализует привязку авторизации к ПК и пользователю Windows через отпечаток системы (серийный номер тома C:\, имя компьютера, имя пользователя ОС), хранение отпечатка и имени в файле в Local AppData и проверку

> при запуске. Токен не переносится на другой компьютер и не подходит под другой учётной записью Windows. Конспект разбирает каждую функцию, параметры Windows API, формат файла и краевые случаи.